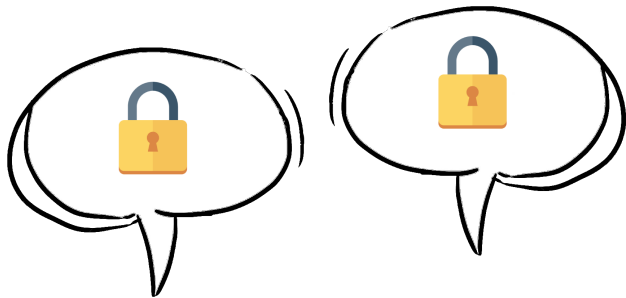


Language models in FHE: from BERT_{tiny} to new challenges

Lorenzo Rovida
Polytechnic University of Turin



Technology Innovation Institute (TII) — Apr 28th, 2026

Agenda

1. Introduction

1.1. Motivations

1.2. Fully homomorphic encryption

1.3. Language models

2. A possible solution: [RL24]

2.1. The main tools

2.2. The encrypted circuit

2.3. Experiments

3. New approaches

3.1. Polynomial Transformers

3.2. Llama-2-7B in CKKS

4. Future challenges

Motivations

- ▶ According to the terms of use of a *very famous* LLM service:

We retain certain data from your interactions with us, but we take steps to reduce the amount of personal information in our training datasets before they are used to improve and train our models.

Motivations

- According to the terms of use of a **very famous** LLM service:

We retain certain data from your interactions with us, but we take steps to reduce the amount of personal information in our training datasets before they are used to improve and train our models.



Motivations

- ▶ A possible solution would be to send user data in an encrypted state, without giving the service provider the key to decrypt
- ▶ How can the service provider still be able to offer the service?

Fully homomorphic encryption

- ▶ *Fully homomorphic encryption* (FHE) is a cryptographic primitive that allows to encrypt data in such a way that operations performed on ciphertexts are reflected as the same operations on corresponding plaintexts.

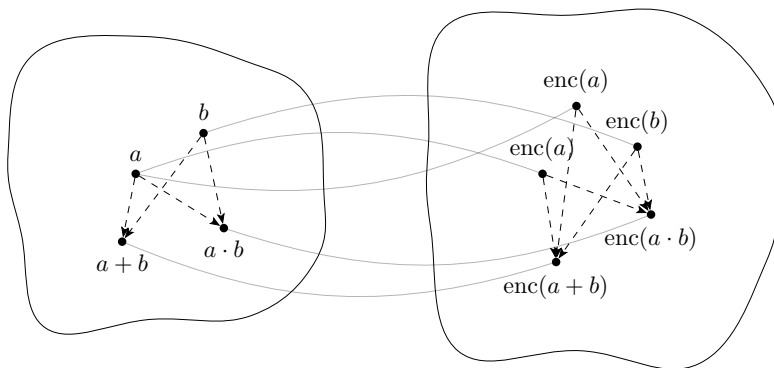
$$a + b = \text{dec}(\text{enc}(a) + \text{enc}(b))$$

$$a \times b = \text{dec}(\text{enc}(a) \times \text{enc}(b))$$

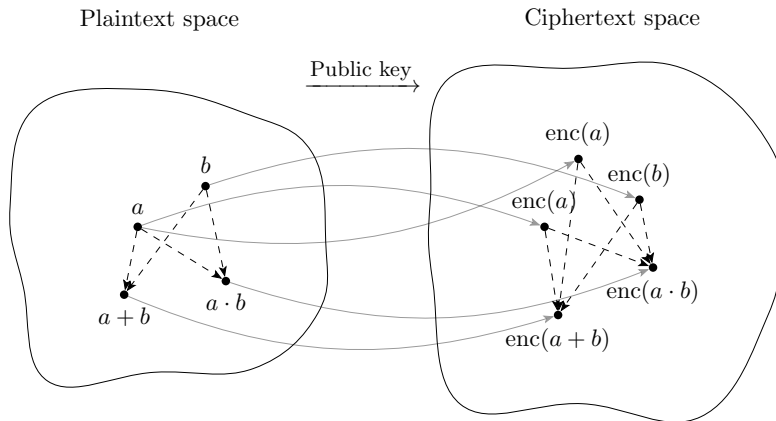
Fully homomorphic encryption

Plaintext space

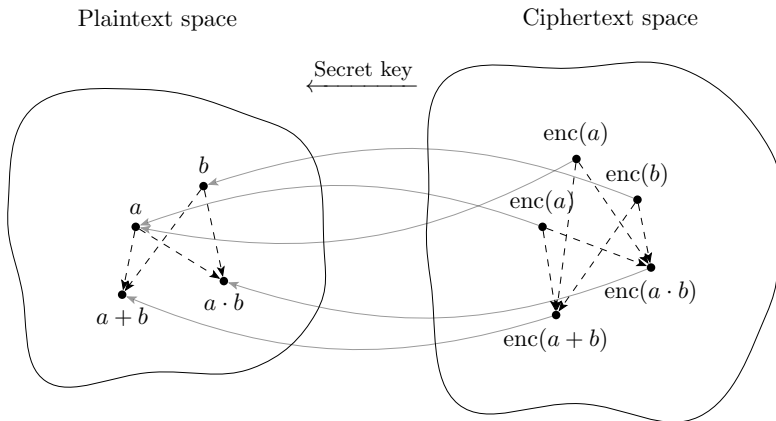
Ciphertext space



Fully homomorphic encryption



Fully homomorphic encryption



Fully homomorphic encryption

Research question, in general

How efficient is to evaluate a Large language model in the ciphertext domain? Which are the main bottlenecks?

The CKKS scheme

- ▶ In literature, we can find different FHE schemes which mainly differ by their plaintext domain (i.e., exact or approximate) and their packing properties (i.e., single value vs SIMD)
- ▶ In most LLM implementations, the CKKS ([CKKS17]) scheme is used as it provides a large throughput combined with (approximate) real numbers arithmetic.

[CKKS17]: J.H. Cheon et al. “Homomorphic Encryption for Arithmetic of Approximate Numbers”. In: *ASIACRYPT '17*. 2017

The CKKS scheme, in a nutshell

Given $N = 2^k$, the encoding phase is defined as follows:

- ▶ Given some message $\mathbf{m} \in \mathbb{C}^{N/2}$, compute an inverse DFT such that $\text{iDFT} : \mathbb{C}^{N/2} \rightarrow \mathbb{R}[X]/(X^N + 1)$
- ▶ Use a large scale $\Delta > 0$ to mimic a fixed-point representation of the input
- ▶ In order to use RLWE encryption, round the polynomial in $\mathbb{R}[X]/(X^N + 1)$ to $\mathbb{Z}[X]/(X^N + 1)$

$$\text{Encode}(\mathbf{m}) := \lfloor \Delta \cdot \text{iDFT}(\mathbf{m}) \rfloor \in \mathbb{Z}[X]/(X^N + 1)$$

The CKKS scheme, in a nutshell

- Notice that the DFT induces a ring homomorphism that enables slot-wise additions and multiplications! Given two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{C}^{N/2}$:

$$\text{DFT}(\text{iDFT}(\mathbf{a}) + \text{iDFT}(\mathbf{b})) = \mathbf{a} + \mathbf{b} \quad \text{and}$$

$$\text{DFT}(\text{iDFT}(\mathbf{a}) \cdot \text{iDFT}(\mathbf{b})) = \mathbf{a} \odot \mathbf{b}$$

- Hence by multiplying two encodings of $\mathbf{m}_1, \mathbf{m}_2$, we are performing a slot-wise multiplication (i.e., $(m_1)_i \cdot (m_2)_i$) of the underlying messages!

The CKKS scheme, in a nutshell

The decoding phase is as the inverse of the encoding

- ▶ Given some encoded message $e \in \mathbb{Z}[X]/(X^N + 1)$, a DFT such that $\text{DFT} : \mathbb{R}[X]/(X^N + 1) \rightarrow \mathbb{C}^{N/2}$.
- ▶ Multiply by the inverse $1/\Delta$.
- ▶ Since we previously rounded with $\lfloor \cdot \rfloor$, some digits might be lost, this is controlled by the magnitude of Δ .

$$\text{Decode}(e) := \frac{\text{DFT}(e)}{\Delta} \in \mathbb{C}^{N/2}$$

The CKKS scheme, in a nutshell

- ▶ After each multiplication, the scale of the resulting polynomial will double from Δ to Δ^2 .
- ▶ The RLWE modulus Q is built as a *moduli ladder*, that is

$$Q = q_1 \cdot q_2 \cdot \dots \cdot q_\ell = \prod_{i=1}^{\ell} q_i$$

Thus we fix $\Delta \approx q_i$, and divide by $1/\Delta$ after each multiplication to restore the scale.

- ▶ Problem: after $\ell - 1$ multiplications, there are no more moduli to “remove”!

The CKKS scheme, in a nutshell

The solution is to perform a *bootstrapping*¹. The main idea is as follows:

1. Homomorphically evaluate the DFT to bring the message from *slots* to *coeffs*.
Now the plaintext contains Δm .
2. Increase the modulus back to Q , this transforms the underline message from Δm to $\Delta m + kq_1$, for some small poly $k(X)$.
3. Homomorphically evaluate the iDFT to go back to slots. Now we encode $\text{iDFT}(\Delta m + kq_1)$, same as before but with the kq_1 term.
4. Evaluate a modular reduction function to remove the kq_1 term (using an approximation of a sin-like function).

¹It is folklore that, in general, bootstrapping in FHE means to homomorphically evaluate the decryption circuit to decrease the noise from $m + e$ to $m + e'$, with $e' < e$. Although this is typically not explicated, this is not the case of CKKS, as the error e in $m + e$ can not be removed as it is part of the message

BERT

- ▶ The Bidirectional Encoder Representations from Transformers (BERT) model was introduced in 2018 by Google [DCLT19].
- ▶ It is usually used to extract features from sentences and to use these features with a classifier (in simple terms: it turns text into numbers:))
- ▶ E.g., Google search uses BERT to understand search queries better, but it is heavily used in many applications (Microsoft Office Suite, Facebook, Alexa, Grammarly...)

[DCLT19]: J. Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *NAACL '19*. June 2019

BERT

- ▶ Despite its complex architecture, behind the Veil of Maya we find... a (somewhat) simple neural network!



BERT

- ▶ It turns out that almost all operations in a neural network are additions and multiplications... so we can rewrite them using FHE!
- ▶ Main issue 1: activation functions can not be shaped using additions and multiplications!
- ▶ Main issue 2: language models in particular are very deep, we have to be careful as the RLWE noise increases at every operation.

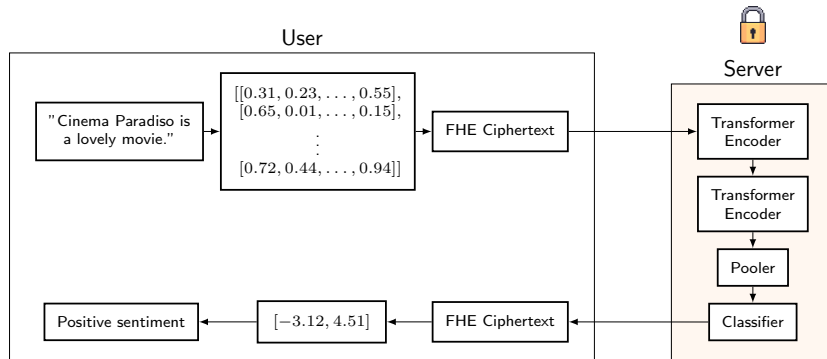
A possible solution: [RL24]

- ▶ In [RL24], they suggested a solution for the evaluation of BERT_{tiny}, the smallest BERT model, using FHE operators only.
- ▶ We will first describe and analyze this proposal, which back in 2024 was the first “practical” solution completely based in FHE.

[RL24]: L. Rovida and A. Leporati. “Transformer-based Language Models and Homomorphic Encryption: An Intersection with BERT-tiny”. In: *IWSPA '24*

A possible solution: [RL24]

- The setting is where the server is *honest-but-curious*, and the client asks for a classification of a sentence without revealing her query.



A possible solution: [RL24]

- ▶ The BERT_{tiny} architecture is composed of $L = 2$ encoders, each with $H = 128$ hidden units.
- ▶ Each encoder contains the following:
 1. Self-attention: computes the attention function, $\text{softmax}(QK^T / \sqrt{d_k}) \cdot V$.
 2. Self-output: computes a linear transformation followed by a normalization
 3. Intermediate: computes a linear transformation and applies the $\text{GeLU}(x)$ function
 4. Output: computer a linear transformation and normalization.

Vector–matrix multiplications

- ▶ In FHE, reshaping the values in a ciphertext is *very costly*.
- ▶ Instead of adapting data to procedures, we do the opposite: adapt procedures to data
- ▶ We therefore use two different algorithms to perform vector–matrix multiplication, depending on how the input is encoded.

Vector-matrix multiplications

- ▶ We will use two different encodings for an input v :
- ▶ *Repeated* encoding:

$$(v_1, v_2, \dots, v_n \mid v_1, v_2, \dots, v_n \mid \dots) \in \mathbb{R}^{n^2}$$

- ▶ *Expanded* encoding:

$$(v_1, v_1, \dots, v_1 \mid v_2, v_2, \dots, v_2 \mid \dots) \in \mathbb{R}^{n^2}$$

Vector-matrix multiplications

- Repeated vector $\times$ column-wise matrix $\rightarrow$ expanded vector

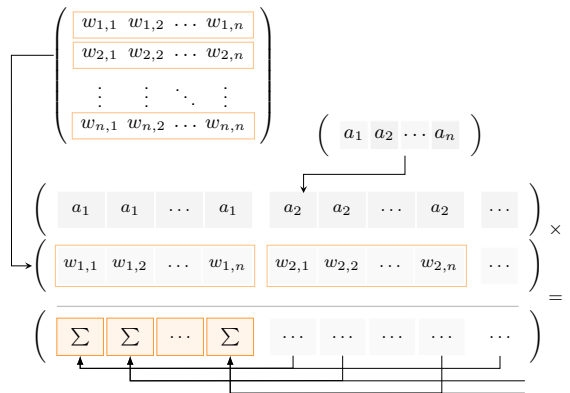
$$\begin{pmatrix} w_{1,1} & w_{1,2} & \dots & w_{1,n} \\ w_{2,1} & w_{2,2} & \dots & w_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n,1} & w_{n,2} & \dots & w_{n,n} \end{pmatrix} \begin{pmatrix} a_1 & a_2 & \dots & a_n \end{pmatrix}$$

$$\begin{pmatrix} a_1 & a_2 & \dots & a_n & a_1 & a_2 & \dots & a_n & \dots \end{pmatrix} \times \begin{pmatrix} w_{1,1} & w_{2,1} & \dots & w_{n,1} & w_{1,2} & w_{2,2} & \dots & w_{n,2} & \dots \end{pmatrix} = \begin{pmatrix} \sum & \dots & \dots & \dots & \sum & \dots & \dots & \dots & \dots \end{pmatrix}$$

The diagram illustrates the expansion of a vector-matrix multiplication. The first part shows a matrix with columns $w_{1,1}, w_{1,2}, \dots, w_{1,n}$ and a vector $(a_1, a_2, \dots, a_n)$. The second part shows the resulting expanded vector, which is the product of the first row of the matrix and the vector, followed by the product of the second row and the vector, and so on. The final part shows the resulting vector, which is the sum of the products of the first row and the vector, followed by the sum of the products of the second row and the vector, and so on.

Vector-matrix multiplications

- Expanded vector $\times$ row-wise matrix $\rightarrow$ repeated vector



Vector–matrix multiplications

- ▶ Each token in a sentence is a vector, so we will have plenty of them!
- ▶ We want to pack them, so that we usually bootstrap a single ciphertext.
- ▶ Instead of resorting to an actual (matrix $\times$ matrix) multiplication, we parallelize the different vector–matrix multiplications and we wrap the results up².

²Different approaches to matrix $\times$ matrix are possible, including baby–step–giant–step algorithms. The presented approach is very simple and with $\Theta(\log(n))$ rotations, and is highly parallelizable

Vector-matrix multiplications

► If the results are repeated:

$$a : \left(\boxed{a_1} \mid a_1 \mid a_1 \mid \boxed{a_2} \mid a_2 \mid a_2 \mid \boxed{a_3} \mid a_3 \mid a_3 \right)$$

$$b : \left(b_1 \mid \boxed{b_1} \mid b_1 \mid b_2 \mid \boxed{b_2} \mid b_2 \mid b_3 \mid \boxed{b_3} \mid b_3 \right)$$

$$c : \left(c_1 \mid c_1 \mid \boxed{c_1} \mid c_2 \mid c_2 \mid \boxed{c_2} \mid c_3 \mid c_3 \mid \boxed{c_3} \right)$$

$$M : \left(\boxed{a_1} \mid \boxed{b_1} \mid \boxed{c_1} \mid \boxed{a_2} \mid \boxed{b_2} \mid \boxed{c_2} \mid \boxed{a_3} \mid \boxed{b_3} \mid \boxed{c_3} \right)$$

Vector-matrix multiplications

► If the results are expanded:

$$a : \left(\begin{array}{|c|c|c|} \hline a_1 & a_2 & a_3 \\ \hline \end{array} \quad \begin{array}{|c|c|c|} \hline a_1 & a_2 & a_3 \\ \hline \end{array} \cdot \quad \begin{array}{|c|c|c|} \hline a_1 & a_2 & a_3 \\ \hline \end{array} \right)$$

$$b : \left(\begin{array}{|c|c|c|} \hline b_1 & b_2 & b_3 \\ \hline \end{array} \quad \begin{array}{|c|c|c|} \hline b_1 & b_2 & b_3 \\ \hline \end{array} \cdot \quad \begin{array}{|c|c|c|} \hline b_1 & b_2 & b_3 \\ \hline \end{array} \right)$$

$$c : \left(\begin{array}{|c|c|c|} \hline c_1 & c_2 & c_3 \\ \hline \end{array} \quad \begin{array}{|c|c|c|} \hline c_1 & c_2 & c_3 \\ \hline \end{array} \cdot \quad \begin{array}{|c|c|c|} \hline c_1 & c_2 & c_3 \\ \hline \end{array} \right)$$

$$M : \left(\begin{array}{|c|c|c|} \hline a_1 & a_2 & a_3 \\ \hline \end{array} \quad \begin{array}{|c|c|c|} \hline b_1 & b_2 & b_3 \\ \hline \end{array} \cdot \quad \begin{array}{|c|c|c|} \hline c_1 & c_2 & c_3 \\ \hline \end{array} \right)$$

Activation functions

- ▶ Remark that in CKKS we can only evaluate additions and multiplications
- ▶ In a Transformer architecture we have many activations and non polynomial functions: $\text{GELU}(x)$, $1/x$, e^x , etc.

Activation functions

- Luckily we have the following theorem:

Theorem (Chebyshev roots are near-best [Tre13])

Let $f(x)$ be a continuous function in $[1, 1]$, let $p(x)$ denote its interpolant in n Chebyshev points and let $p^*(x)$ be its best degree n approximation with respect to the infinity norm. It holds that:

$$\|f(x) - p(x)\|_{\infty} \leq \|f(x) - p^*(x)\| \cdot \left(2 + \frac{2}{\pi} \log(n)\right)$$

[Tre13]: L.N. Trefethen. *Approximation Theory and Approximation Practice*. Other Titles in Applied Mathematics. Siam, 2013

Activation functions

- ▶ Chebyshev polynomials are flexible and give a very good approximation.
- ▶ We already have algorithms (such as the one described in [CCS19]) to evaluate them quickly.
- ▶ Remark that these approximations work in a fixed interval $[a, b]$, thus an empirical study on the input values must be performed.

[CCS19]: H. Chen, I. Chillotti, and Y. Song. “Improved Bootstrapping for Approximate Homomorphic Encryption”. In: *EUROCRYPT 2019*

Self-attention

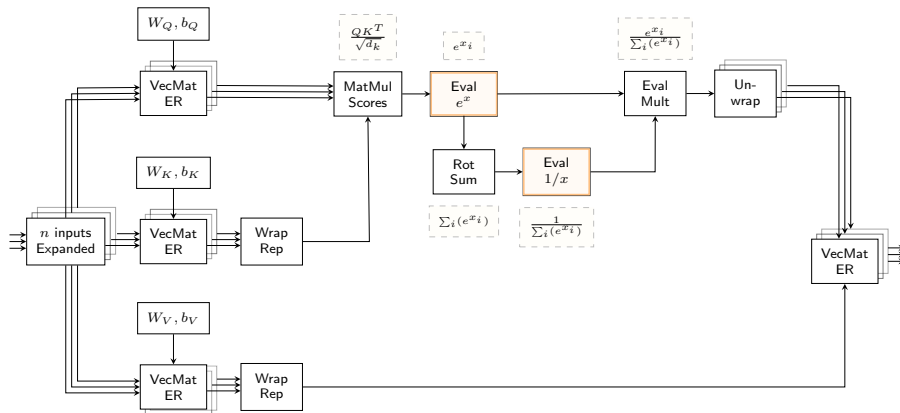
- The first layer computes the following function:

$$\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

where $\text{softmax}(x) = \frac{e^{x_i}}{\sum_j e^{x_j}}$ and $Q, K, V \in \mathbb{R}^{128 \times t}$ are the result of a linear transformation, and t is the number of tokens.

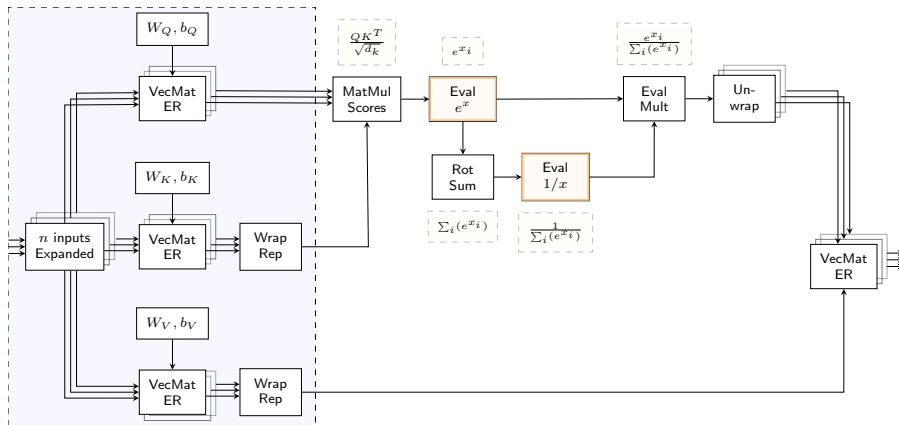
Self-attention

► We can illustrate it as follows:



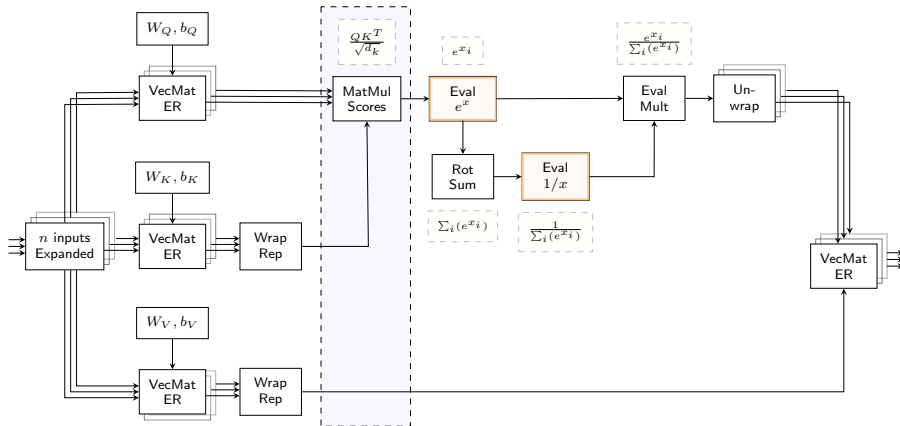
Self-attention

- The first part computes the Q, K, V matrices.



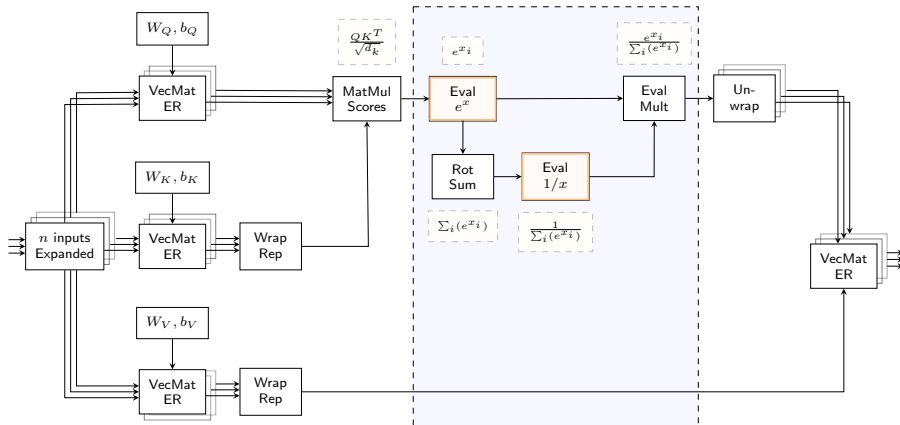
Self-attention

- The second part computes the dot-product attention (so called *scores*).



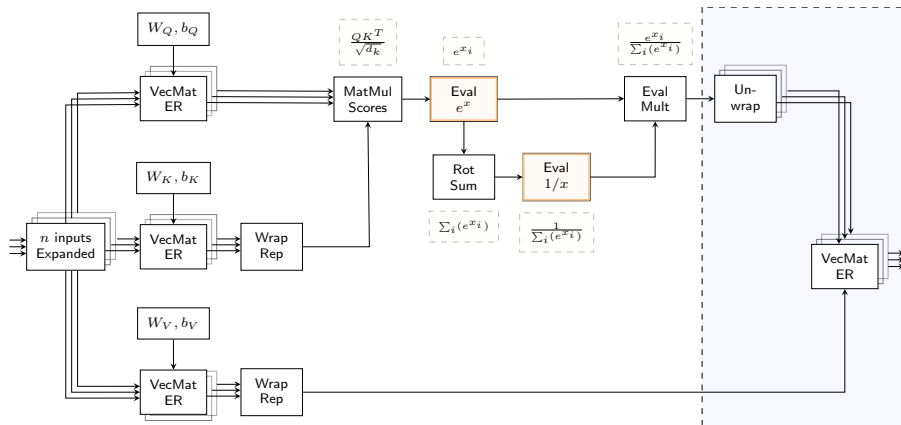
Self-attention

- The softmax function is computed by approximating e^x and $1/x$.



Self-attention

- The product with V concludes the evaluation of the Self-attention.



Self-output

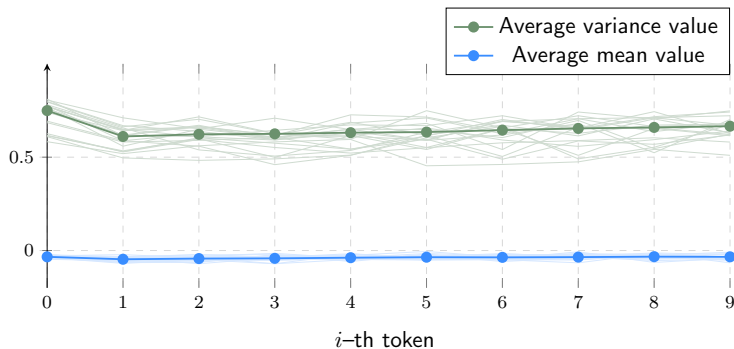
- ▶ The Self-output block computes a linear transformation followed by a Layer normalization, defined as:

$$\text{LayerNorm}(X) = \frac{x_i - \mathbb{E}[X]}{\sqrt{\text{Var}(X) + \varepsilon}} \cdot \gamma + \beta$$

- ▶ where $\mathbb{E}[X]$, $\text{Var}[X]$ are the mean and the variance, resp.
- ▶ However, this would require another approximation ($1/\sqrt{x}$).

Self-output

- ▶ Inspired by ResNet, we propose to skip this approximation, and to use constant values for mean and variance.
- ▶ This makes the layer equivalent to a linear transformation. As a cons, results not be identical as the plain model.



Intermediate

- ▶ The third block, called Intermediate, increases the size of the features from 128 to 512.
- ▶ After that, a $\text{GELU}(x)$ activation function is evaluated:

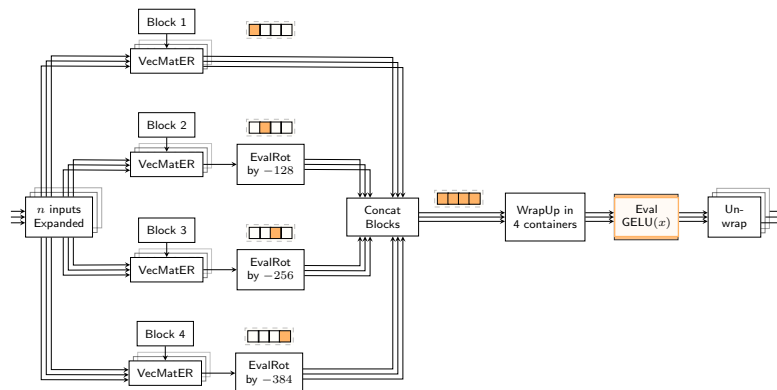
$$\text{GELU}(x) = \frac{x}{2} \left(1 + \text{erf} \left(\frac{x}{\sqrt{2}} \right) \right)$$

where $\text{erf}(x)$ is the error function.

- ▶ Issue: each ciphertext contains up to $128 \cdot 128 = 16384$ values, but the weights are of size $512 \cdot 128$.

Self-attention

- The linear transformation is executed four times ($4 \cdot 128 = 512$), wrapped up in four containers and $\text{GELU}(x)$ is evaluated.

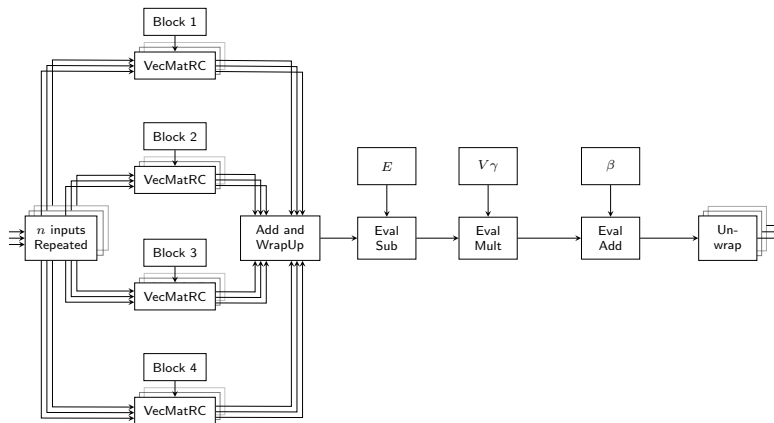


Output

- ▶ The last block, called Output, decreases the size of the features back to 128.
- ▶ After that, a Layer normalization is performed.
- ▶ As before, we use precomputed values for mean and variance.

Output

- Features are decreased back to 128 and a linear transformation is evaluated.



Experiments

- ▶ The circuit has been implemented in the OpenFHE implementation [ABB⁺22] of the RNS-CKKS scheme [KPP22].
- ▶ The scheme is instantiated with the following parameters:
 - ▶ Ring of dimension $N = 2^{16}$;
 - ▶ Ciphertext slots $s = N/4$;
 - ▶ Moduli chain values $q_i = 52$ bits;
 - ▶ Circuit depth $d = 24$;
 - ▶ Ciphertext modulus $Q = 1767$ bits;
 - ▶ Hamming weight of encapsulated secret key (dense h , sparse $\hat{h}$) $h, \hat{h} = (128, 32)$

[ABB⁺22]: A. Al Badawi et al. “OpenFHE: Open-Source Fully Homomorphic Encryption Library”. In: *WAHC '22*

[KPP22]: A. Kim, A. Papadimitriou, and Y. Polyakov. “Approximate Homomorphic Encryption with Reduced Approximation Error”. In: *CT-RSA '22*

Experiments

- In the following we present some parameters of the Chebyshev polynomials, based on experimental observations on the training set.

Layer	Function	Interval	Degree
Encoder 1 – Self-Attention	$1/x$	$[2, 5000]$	119
Encoder 1 – Intermediate	$\text{GELU}(x)$	$[-14, 11]$	119
Encoder 2 – Self-Attention	$1/x$	$[3, 130000]$	200
Encoder 2 – Intermediate	$\text{GELU}(x)$	$[-18, 8]$	59
Pooler	$\tanh(x)$	$[-20, 20]$	300

Experiments

- Accuracy results on the SST-2 validation set for the three networks. LN stands for Layer normalization.

Model	Accuracy	Loss
BERT _{tiny}	0.837	(<i>reference</i>)
BERT _{tiny} with precomputed LN	0.815	0.022
FHE-BERT _{tiny}	0.790	0.047

Wrapping up [RL24]

- ▶ You can find the open-source implementation at github.com/lorenzorovida/FHE-BERT-Tiny!

**FHE-BERT-Tiny**

Public



Source code for the paper "Transformer-based Language Models and Homomorphic Encryption: an intersection with BERT-tiny"



C++

☆ 32

🔗 12

Wrapping up [RL24]

- ▶ [RL24] is interesting as it has been the first *pure FHE* implementation of a Transformer architecture.
- ▶ Many optimizations and improvements can be done, mainly serves as an initial benchmark.
- ▶ Since this work (started four years ago!), *a lot* of new proposals have emerged
- ▶ Let us review some of them!

[RL24]: L. Rovida and A. Leporati. “Transformer-based Language Models and Homomorphic Encryption: An Intersection with BERT-tiny”. In: *IWSPA '24*

Polynomial Transformers

A wonderful work that has the goal to construct polynomial Transformers

Converting Transformers to Polynomial Form for Secure Inference Over Homomorphic Encryption

Itamar Zimerman ^{1,2} Moran Baruch ^{1,3} Nir Drucker ¹ Gilad Ezov ¹ Omri Soceanu ¹ Lior Wolf ²

Abstract

Designing privacy-preserving DL solutions is a major challenge within the AI community. Homomorphic Encryption (HE) has emerged as one of the most promising approaches in this realm, enabling the decoupling of knowledge between a model owner and a data owner. Despite extensive research and application of this technology, primarily in CNNs, applying HE on transformer models has been challenging because of the difficulties in converting these models into a polynomial form. We break new ground by introducing the first polynomial transformer, providing

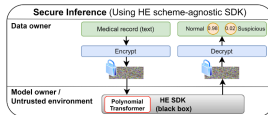


Figure 1: **Schematic Overview of Secure Inference Using HE:** The diagram depicts the sequence of steps where the data owner sends an encrypted sensitive data sample, to the model owner or an untrusted environment. Within this environment, an HE framework (SDK) employs the polynomial

Polynomial Transformers

- ▶ Replace softmax with *pointwise σ -attention* and add *length scaling* to recover stability:

$$\text{Attn}_\sigma(Q, K, V) = \frac{1}{S(L)} \sigma\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad S(L) = \frac{1}{\sqrt{L}}$$

- ▶ This way we avoid (the super heavy) softmax!

Polynomial Transformers

- ▶ Add an auxiliary loss that *shrinks the input domain* of every non-polynomial layer. The training objective $\mathbb{L}$ to be minimized is:

$$\mathbb{L} = \alpha \mathbb{L}_{\text{range}} + \beta \mathbb{L}_{\text{var}} + \mathbb{L}_{\text{task}}$$

- ▶ In practice: large values in Layer normalization ($\mathbb{L}_{\text{var}}$) and at each activation layer ($\mathbb{L}_{\text{range}}$) are *penalized*.
- ▶ *How much accuracy am I willing to sacrifice to keep my internal ranges small?* The α and β hyperparameters control exactly this (large values $\rightarrow$ compress ranges but small accuracy).

○○
○○○○○○○○○○
○○○

○○○
○○○○○○○○○○
○○○○○○○○○○○○
○○○○○

○○○●
○○○○○

○

Polynomial Transformers

- ▶ This is done during a second training phase.
- ▶ By imposing these constraints from the beginning, the model never gets to find a good solution in the first place and can collapse.
- ▶ This reduces variance range from ~ 3300 to ~ 300 , making polynomial approximation more accurate.

Llama-2-7B in CKKS

A very impressive work that implements the huge Llama-2-7B model using CKKS!!

s.CR] 26 Jan 2026

Scaling up Privacy-Preserving ML: A CKKS Implementation of Llama-2-7B

Jaiyoung Park Graduate School of Convergence Science and Technology, Seoul National University Seoul, South Korea	Sejin Park CryptoLab, Inc. & Seoul National University Seoul, South Korea	Jai Hyun Park CryptoLab, Inc. Lyon, France
Jung Ho Ahn Graduate School of Convergence Science and Technology, Seoul National University Seoul, South Korea	Jung Hee Cheon Cryptolab, Inc. & Departement of Mathematics, Seoul National University Seoul, South Korea	Guillaume Hanrot CryptoLab, Inc. Lyon, France
Jung Woo Kim CryptoLab, Inc. Seoul, South Korea	Minje Park CryptoLab, Inc. Seoul, South Korea	Damien Stehlé CryptoLab, Inc. Lyon, France

Abstract

As large language models (LLMs) become ubiquitous, privacy concerns pertaining to inference inputs keep growing. In this context, fully homomorphic encryption (FHE) has emerged as a primary cryptographic solution to provide non-interactive confidential LLM

1 Introduction

When querying a Large Language Models (LLM), clients typically send queries to a computing server that runs the model inference. As these queries and their results often contain sensitive data, privacy concerns emerge. For this reason, various attempts have been made

Outlier mitigation

- ▶ LLMs produce extremely large activation values during inference.
- ▶ In Llama-2-7B, scalar magnitudes can exceed 1000
- ▶ Large activations require high-degree polynomial approximations in FHE circuits
- ▶ Two distinct types of outliers:
 - ▶ *Attention sink outliers*: triggered by special tokens at first processing
 - ▶ *Dimension-wise outliers*: some coordinates are consistently large across all tokens

Outlier mitigation

Attention sinks:

- ▶ Prepend a precomputed key–value cache derived from sink–inducing tokens and stabilizes activation distribution for subsequent tokens.
- ▶ Prefix is query-independent thus no confidential information leaked.

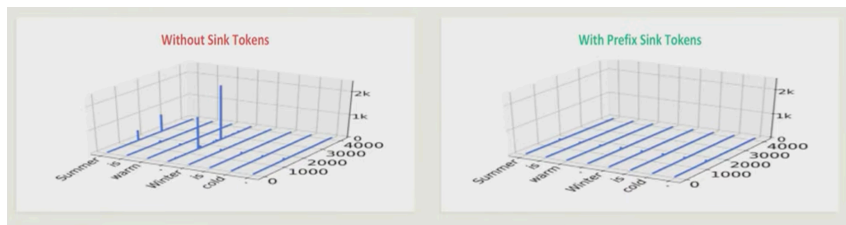


Figure: The impact of prefix sink tokens, credits to J. Park slides at FHE.org 2026

Outlier mitigation

Dimension-wise outliers:

- ▶ Multiply hidden states and weight matrices by orthogonal transformations and redistributes large spikes uniformly across all dimensions. Use $O \in \mathcal{O}_n(\mathbb{R})$ such that $\max|O \cdot x| < \max|x|$.
- ▶ Rotations are fused into adjacent weight matrices $\Rightarrow$ no change to model output

In practice, activation ranges are heavily reduced:

- ▶ RMSNorm: -99.7%
- ▶ SiLU: -53%
- ▶ Softmax: -16%

Llama-7-2B in CKKS

- ▶ Many other optimizations: a novel approach to matrix $\times$ matrix multiplication, log-depth evaluation of polynomial approximations, and so on...
- ▶ On the hardware side, they employed 8x RTX 5090
- ▶ The first token is generated in 15 seconds, the following ones 1 second (context: 128 tokens)

Future challenges

- ▶ We presented a first Transformer implementation in FHE.
- ▶ This is already old! New and fresh proposals enable more efficient and accurate evaluations!

Take home message

Do not try to replicate a clear network in FHE, apply some processing that allow to reduce ranges and heavy activations :) Without *synergy* with the ML community we are lost!

Bibliography I



A. Al Badawi et al. “OpenFHE: Open-Source Fully Homomorphic Encryption Library”. In: *WAHC '22*.



H. Chen, I. Chillotti, and Y. Song. “Improved Bootstrapping for Approximate Homomorphic Encryption”. In: *EUROCRYPT 2019*.



J.H. Cheon et al. “Homomorphic Encryption for Arithmetic of Approximate Numbers”. In: *ASIACRYPT '17*. 2017.



J. Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *NAACL '19*. June 2019.



A. Kim, A. Papadimitriou, and Y. Polyakov. “Approximate Homomorphic Encryption with Reduced Approximation Error”. In: *CT-RSA '22*.

Bibliography II



L. Rovida and A. Leporati. “Transformer-based Language Models and Homomorphic Encryption: An Intersection with BERT-tiny”. In: *IWSPA '24*.



L.N. Trefethen. *Approximation Theory and Approximation Practice*. Other Titles in Applied Mathematics. Siam, 2013.